

GUÍA DE USO BÁSICO

Aitor Ainzúa, Enrique Tomás, Estela Valer



WINDOW BUILDER

Eclipse

LOS DERECHOS DE AUTOR DE ESTE MANUAL

están reservados a:

Aitor Ainzúa, Enrique Tomás y Estela Valer

©2019

Prefacio

El *Manual del usuario de manejo de Window Builder para Java en el IDE de Eclipse* tiene como objetivo responder a las preguntas relativas a la instalación y ejecución de dicho *plugin*. El documento cubre toda la información relacionada con las funciones y opciones, la instalación y configuración, la resolución de problemas y el manejo básico de la herramienta para el manejo de interfaces gráficas en lenguaje Java.

Este manual distribuye la información por capítulos, especialmente diseñados para responder al tipo de preguntas que un usuario se haría al instalar, configurar y utilizar el *plugin* que nos proporciona el IDE de Eclipse. En los encabezamientos de cada capítulo encontraremos las palabras claves necesarias para poder acceder a esa información en concreto sin necesidad de leer todo el documento de esta manera es el usuario es el que determina cuánta información necesita leer.

El índice de contenido que encontrará a continuación permite localizar rápidamente cualquier tema o tarea en particular. En caso de tener todo previamente instalado, recomendamos acudir directamente al punto *Manejo de las funciones básicas*, el cual guiará a modo de tutorial paso a paso como utilizar las herramientas que nos proporciona el *plugin*. Una vez entendido este punto de partida, será realmente sencillo empezar a trabajar con la herramienta de manera autónoma. Los dos últimos puntos le ayudarán a entender algunos términos más específicos o a profundizar más sobre esta herramienta, en caso de que le resulte de interés seguir indagando acerca de *Window Builder* o del IDE de *Eclipse*.

Índice

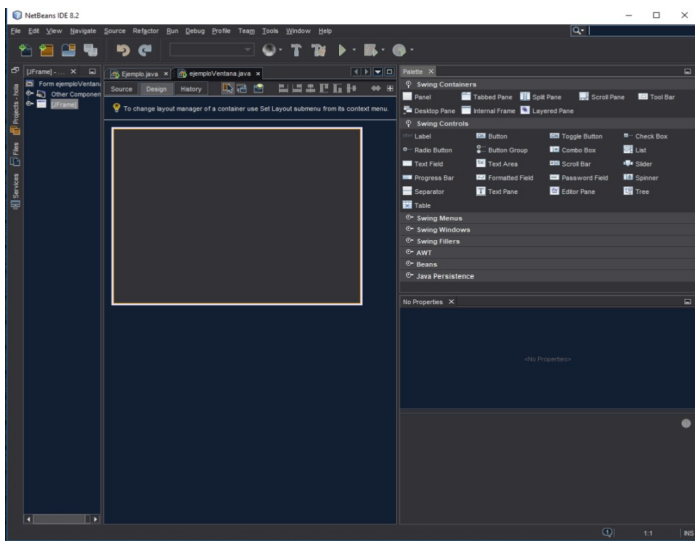
Introducción.....	6
Requisitos previos.....	7
Conocimientos previos.....	7
Requisitos mínimos del sistema.....	7
Mecanismos para acceder al sistema.....	8
Instalación y configuración.....	8
Manejo de las funciones básicas.....	12
Solución de problemas y preguntas frecuentes.....	33
Glosario.....	34
Webgrafía.....	35

Introducción

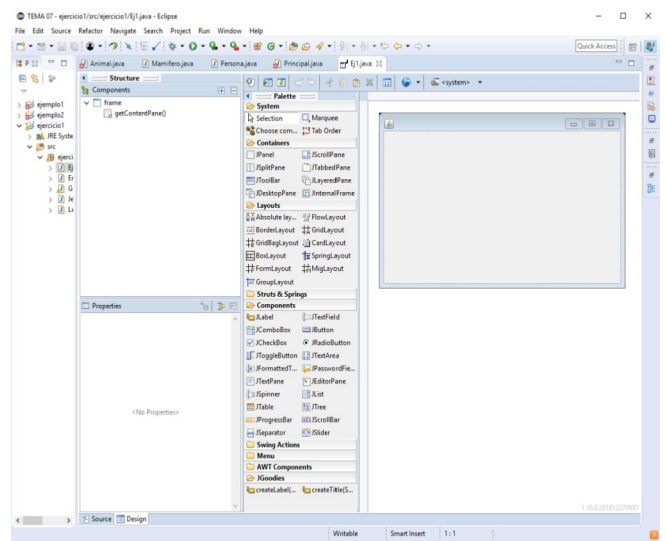
WindowBuilder fue comercializado por la empresa Instantiations, quien fue comprada por Google, el cual lo terminó donando a Eclipse Foundation y convirtiéndolo en código abierto, así pues, *Window Builder* es un plugin para el IDE de Eclipse, el cual nos facilita el trabajo con interfaces gráficas gracias a la paleta de elementos que nos proporciona. Simplemente, seleccionando el elemento que queramos utilizar (un botón, un cuadro de texto...) el propio IDE nos generará automáticamente la base del código sin tener que descargarnos ningún tipo de librería específica. Además, no será necesario compilar el programa para poder ver el resultado final de nuestro programa, sino que podremos ver y testear nuestro código del mismo modo que cuando programamos usando la salida por consola.

Hasta esta última versión la interfaz gráfica que nos proporcionaba eclipse era bastante poco intuitiva y difícil de manejar si nunca habías trabajado con ventanas, otras opciones como NetBeans eran mucho más apropiadas para este tipo de usuarios. En cambio, la última versión de Window Builder nos recuerda bastante en materia de manejabilidad a la herramienta proporcionada de serie por NetBeans, por lo que si utilizas Eclipse como tu entorno de desarrollo habitual puedes seguir en él sin necesidad de tener que importar o exportar proyectos de un IDE a otro. Aquí vemos una comparación visual de ambos entornos.

Interfaz gráfica de Netbeans:



Interfaz gráfica de Eclipse:



Requisitos previos

Conocimientos previos

Para poder manejar las interfaces gráficas es necesario tener una base suficientemente sólida de programación en lenguaje Java, tratado tanto de manera estructurada, utilizando métodos; como orientada a objetos y a eventos; que son las clases y qué son las interfaces. ya que todos los elementos gráficos serán tratados como objetos y deberás conocer cómo trabajar con ellos. Así mismo, será conveniente que conozcamos cómo y por qué motivos se pueden y/o deben tratar las excepciones, ya que una de las primeras líneas de código que aparecen al abrir la interfaz es el tratamiento de la excepción que surgirá en caso de que no consiga abrirse la ventana al ejecutar el código.

Requisitos mínimos del sistema

Si no tenemos ninguna versión del IDE de Eclipse instalada en nuestro ordenador ese será el primer paso a seguir. Dependiendo de la versión de Eclipse que queramos instalar necesitaremos instalar una versión u otra de java junto con su jdk correspondiente. La versión que usaremos como base para este manual será la *Oxygen*, una de las cuales nos permitirá trabajar con la última versión del *Window Builder*, mucho más manejable que las anteriores. Para trabajar con esta versión de Eclipse necesitaremos instalar previamente Java 8, junto con el jdk 1.8. Una vez tengamos instalado el Java 8 e instalado y configurado el IDE de Eclipse podremos pasar al siguiente paso del manual para instalar y configurar este plugin.

ADDITIONAL DETAILS 

Eclipse Versions:
2018-09 (4.9), Photon (4.8), Oxygen (4.7), Neon (4.6), 2018-12 (4.10),
2019-03 (4.11)

Platform Support: Windows, Mac, Linux/GTK

Organization Name: Eclipse Foundation

Date Created: Thu, 2016-09-22 09:02

Development Status: Production/Stable

License: EPL

Date Updated: Mon, 2019-01-21 09:15

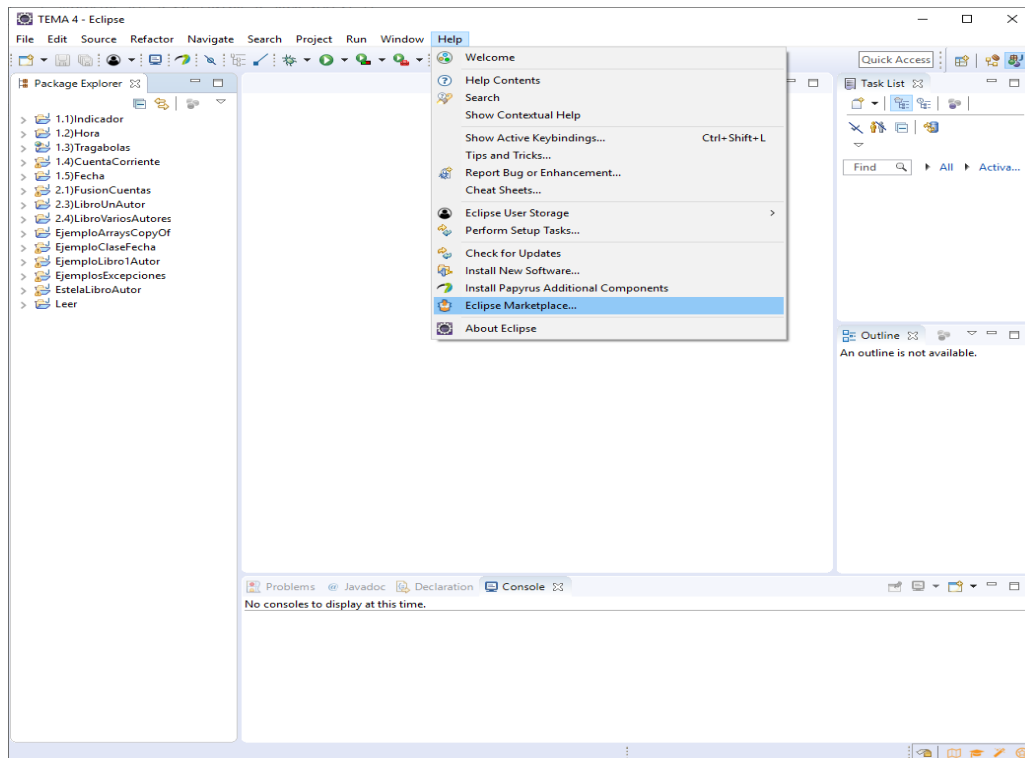
Submitted by: Holger Voormann

Mecanismos para acceder al sistema

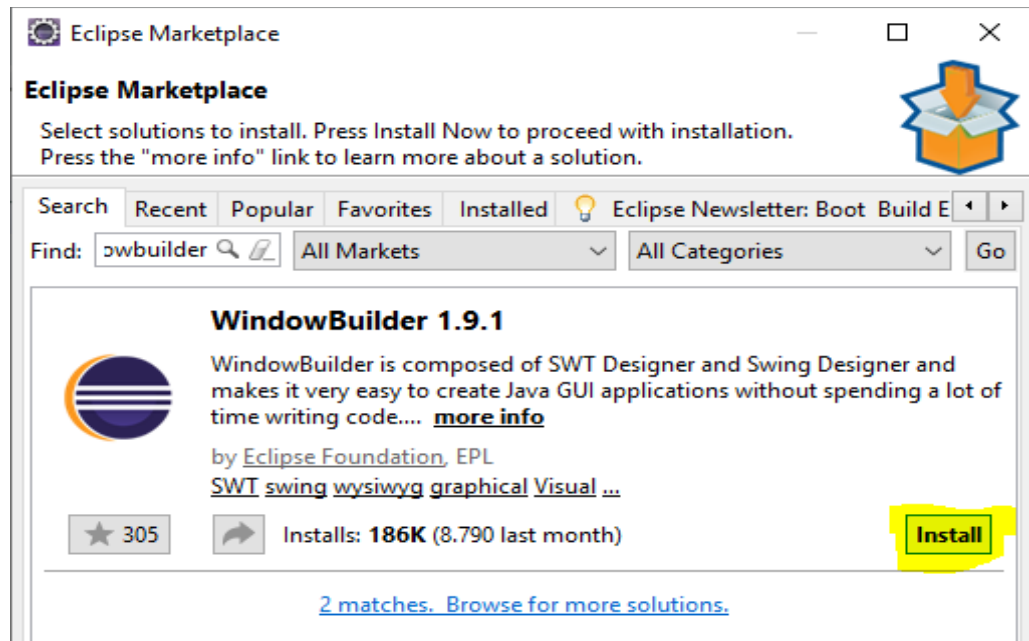
Una vez tengamos instalado y configurado nuestro IDE, deberemos crear una carpeta en nuestro ordenador que nos servirá de *workspace*, es decir, un lugar donde Eclipse guardará todos los archivos necesarios para poder almacenar los proyectos que creemos ahí. Una vez creada la carpeta simplemente deberemos iniciar la aplicación, seleccionar la carpeta que hemos creado para destinarla como nuestro primer *workspace*, crear un proyecto de Java (esto lo podremos hacer también una vez instalado el plugin) y seguir los pasos que detallamos en el siguiente capítulo del manual. Aquí os dejamos un enlace por si encontráis duda en la instalación y configuración del IDE: <https://jarroba.com/instalar-bien-eclipse-un-ide-de-muchos/>.

Instalación y configuración

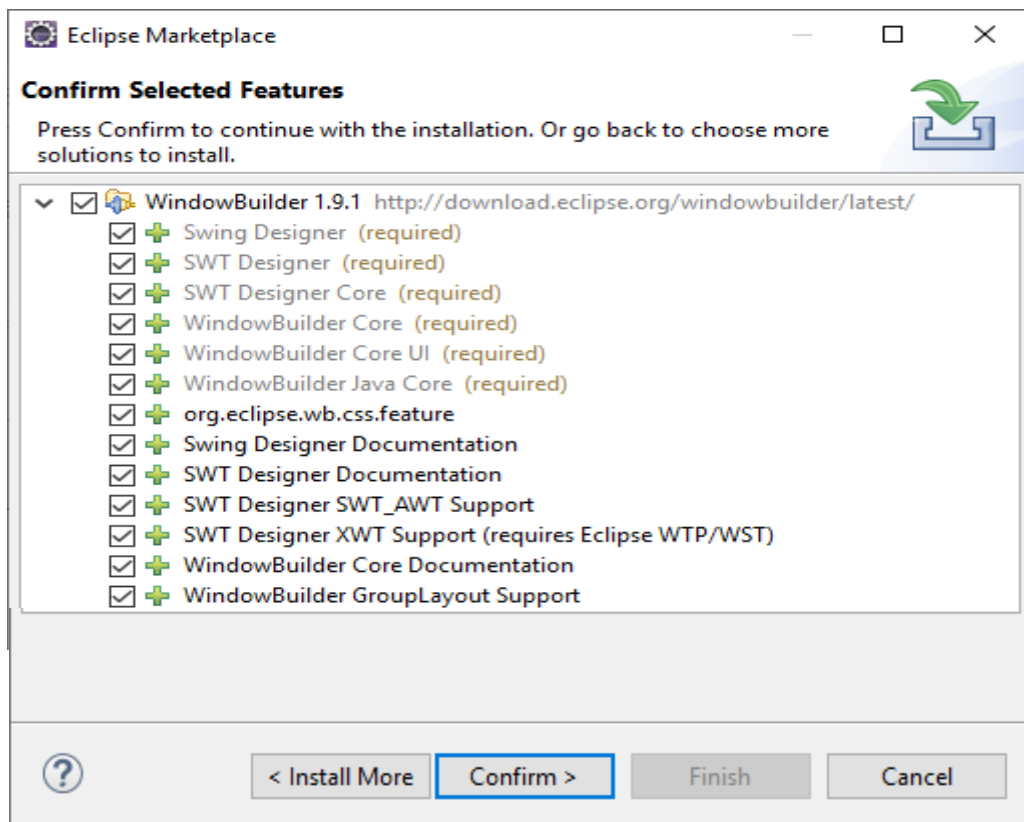
1º. Accedemos al menú Help, dentro del cual podremos encontrar el *Eclipse Marketplace*:



2º. Una vez hayamos accedido al *Marketplace* en la barra de búsqueda escribimos *WindowBuilder* y en el resultado que nos devuelva le damos a *Install*:

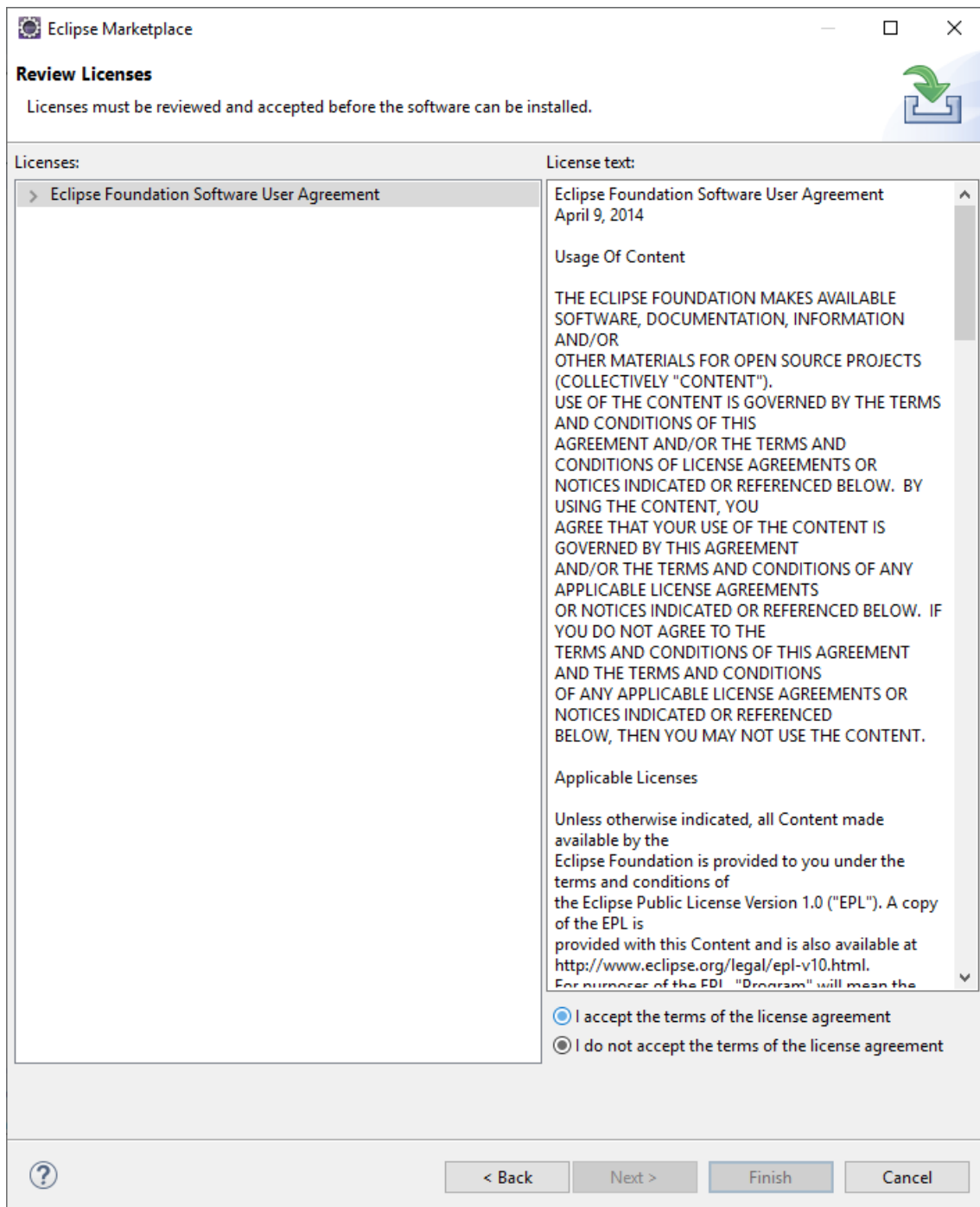


3º. En el siguiente paso nos pedirán que marquemos las características a instalar.



Marcaremos todas ellas y confirmaremos.

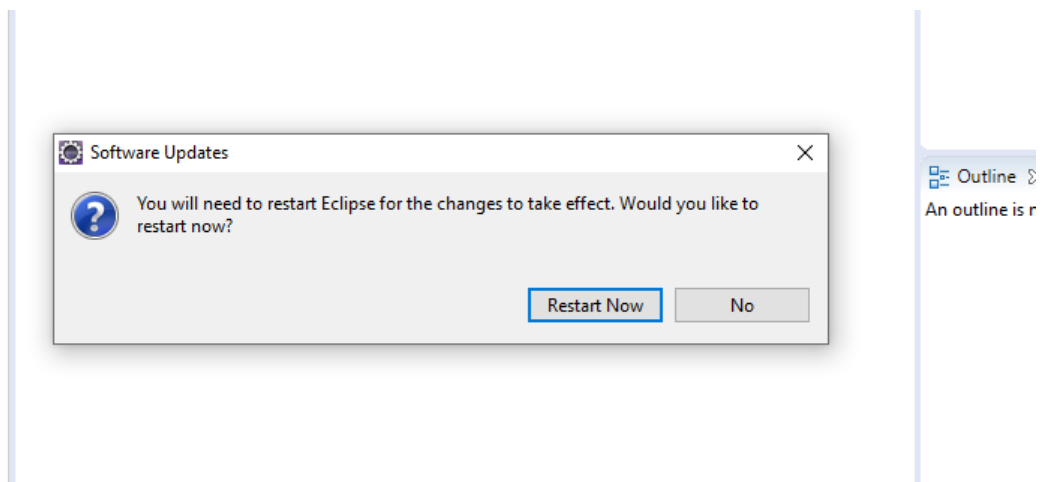
4º. Aceptamos los términos y condiciones de uso:



5°. Una vez aceptadas empezará la instalación, cuyo progreso se nos marcará en la parte inferior derecha de la pantalla principal de nuestro IDE:



6°. Por último tendremos que reiniciar nuestro IDE para que se apliquen los cambios y la instalación del plugin se aplique correctamente:

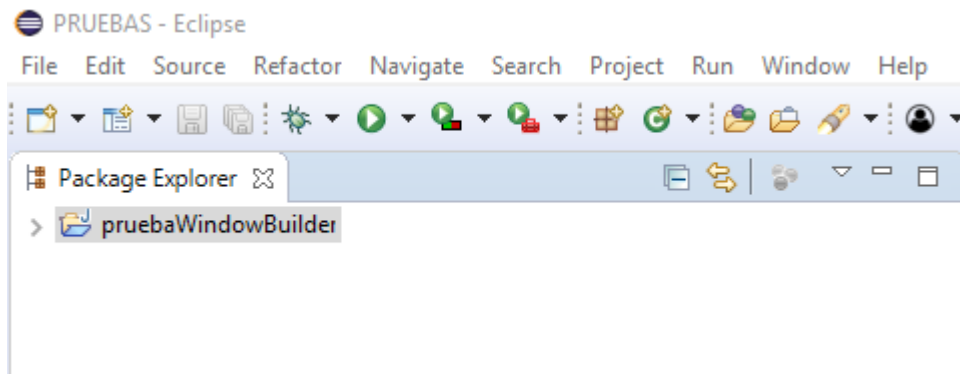


7°. Finalmente, para comprobar que nuestro plugin está correctamente instalado accedemos a *Help* y, a continuación, accedemos a *About Eclipse* en el menú. Ahí comprobaremos que se encuentra entre nuestra lista de plugins instalados:



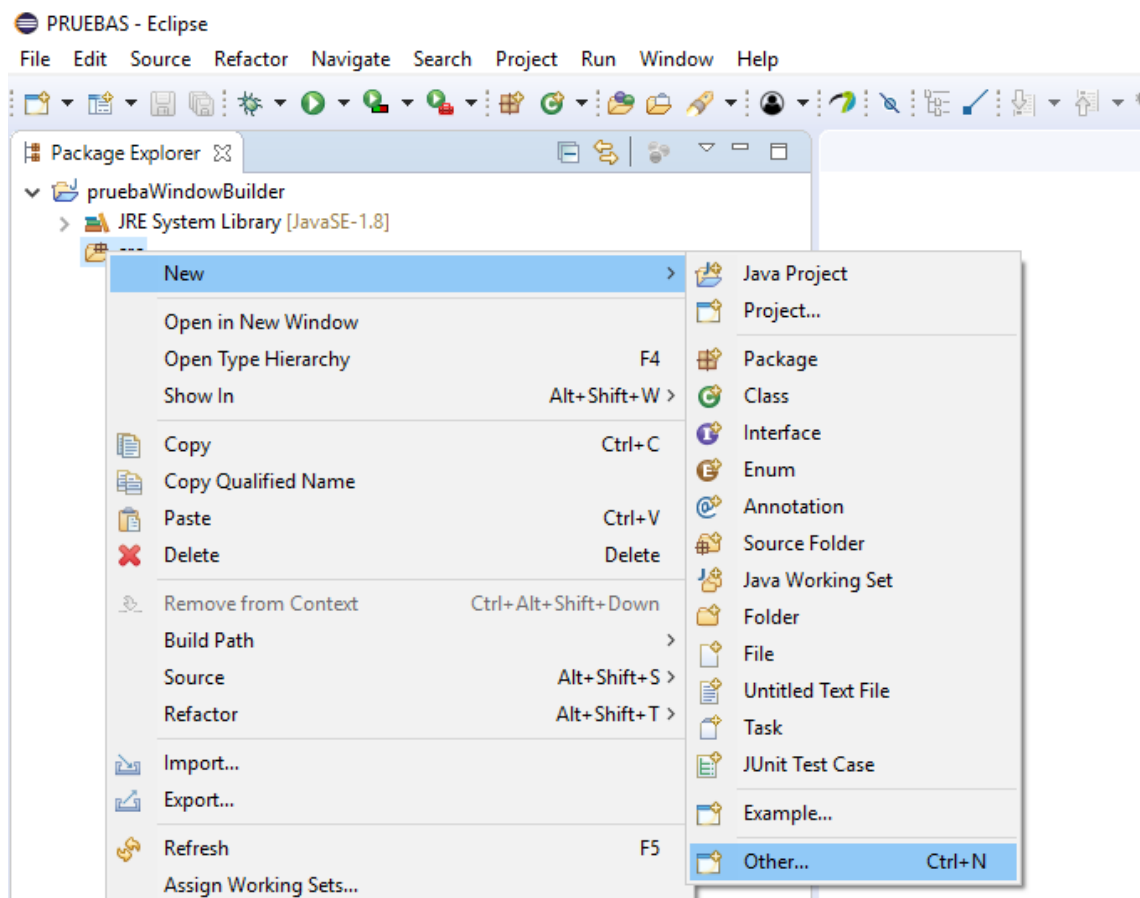
Manejo de las funciones básicas

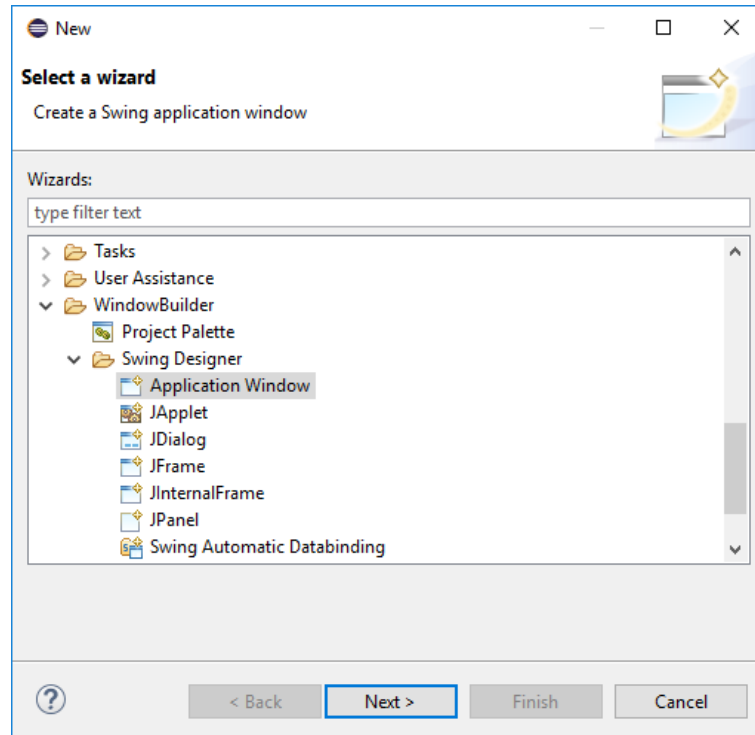
Creación del entorno gráfico:



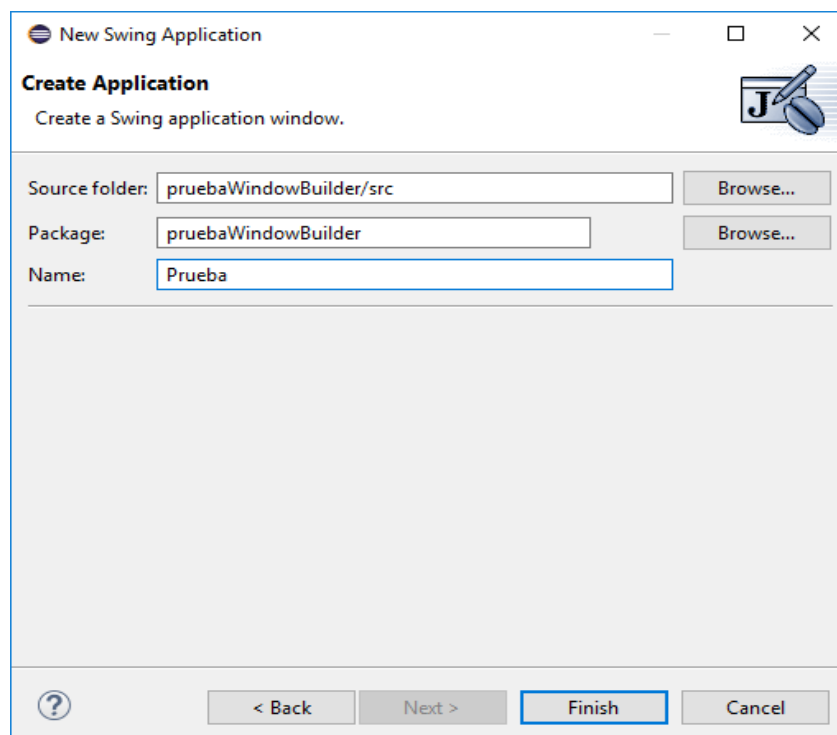
Creamos un nuevo Java Project, el cual lo llamo "pruebaWindowBuilder".

Una vez creado, creamos una clase nueva. Vamos a "new" y dentro de este a "other".



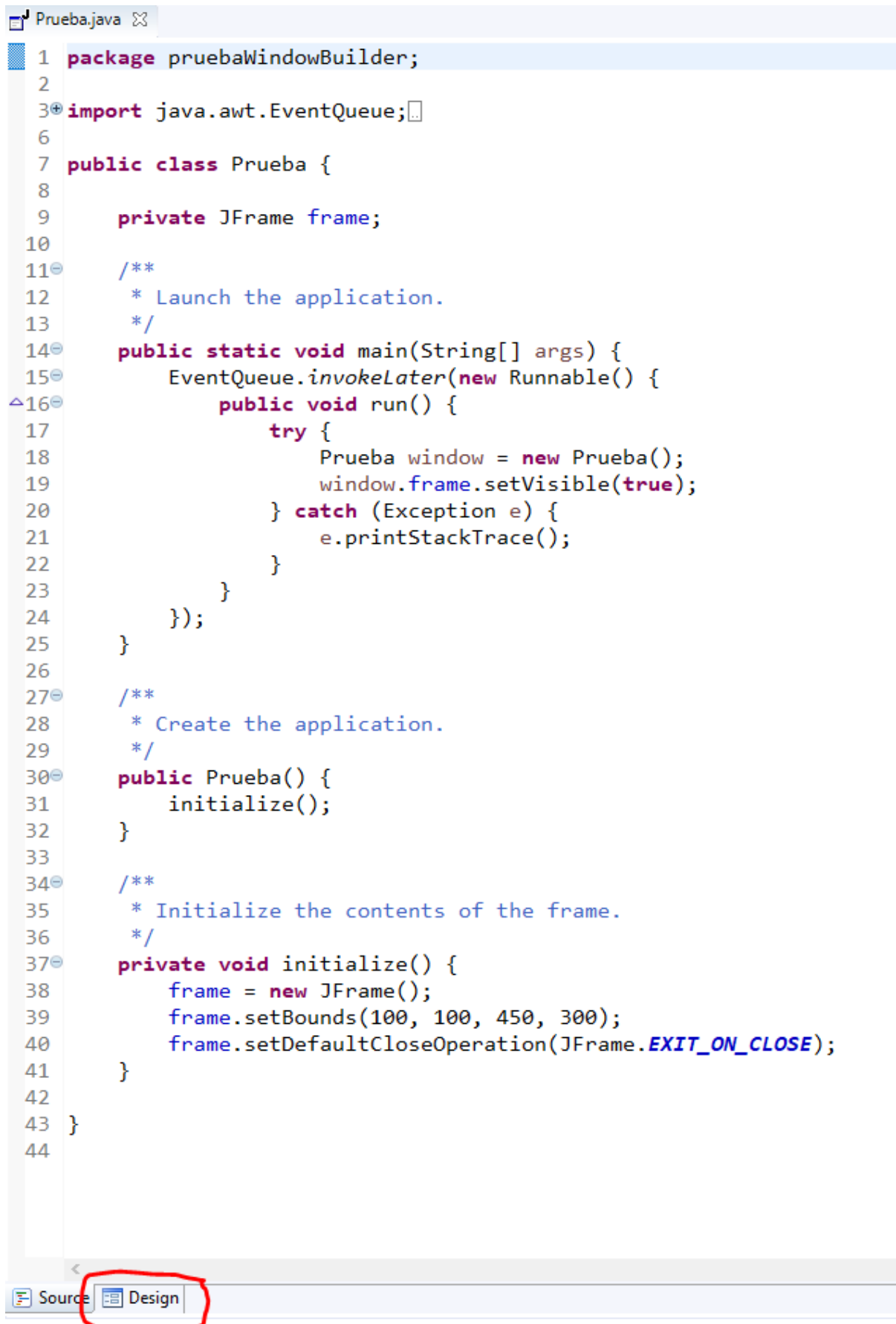


Recordar que el nombre tiene que ir con la primera letra en mayúscula según los estándares de Java ya que es una clase.



Observamos que se nos genera el código automáticamente.

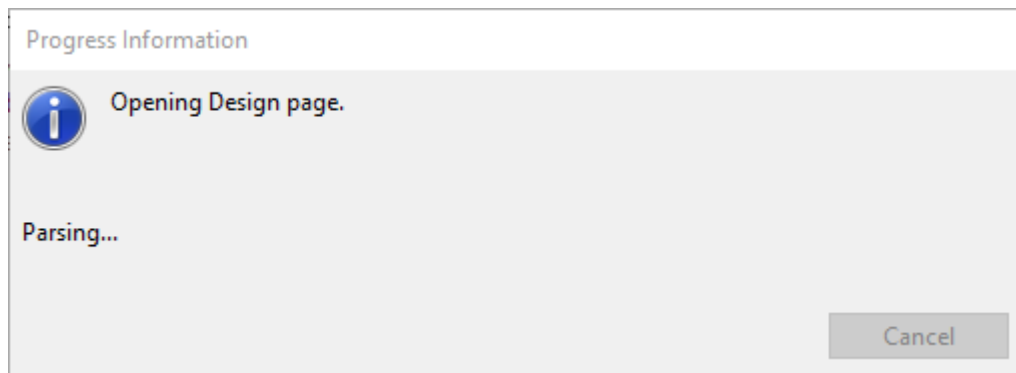
Si vamos a la pestaña design veremos gráficamente lo que es el código.



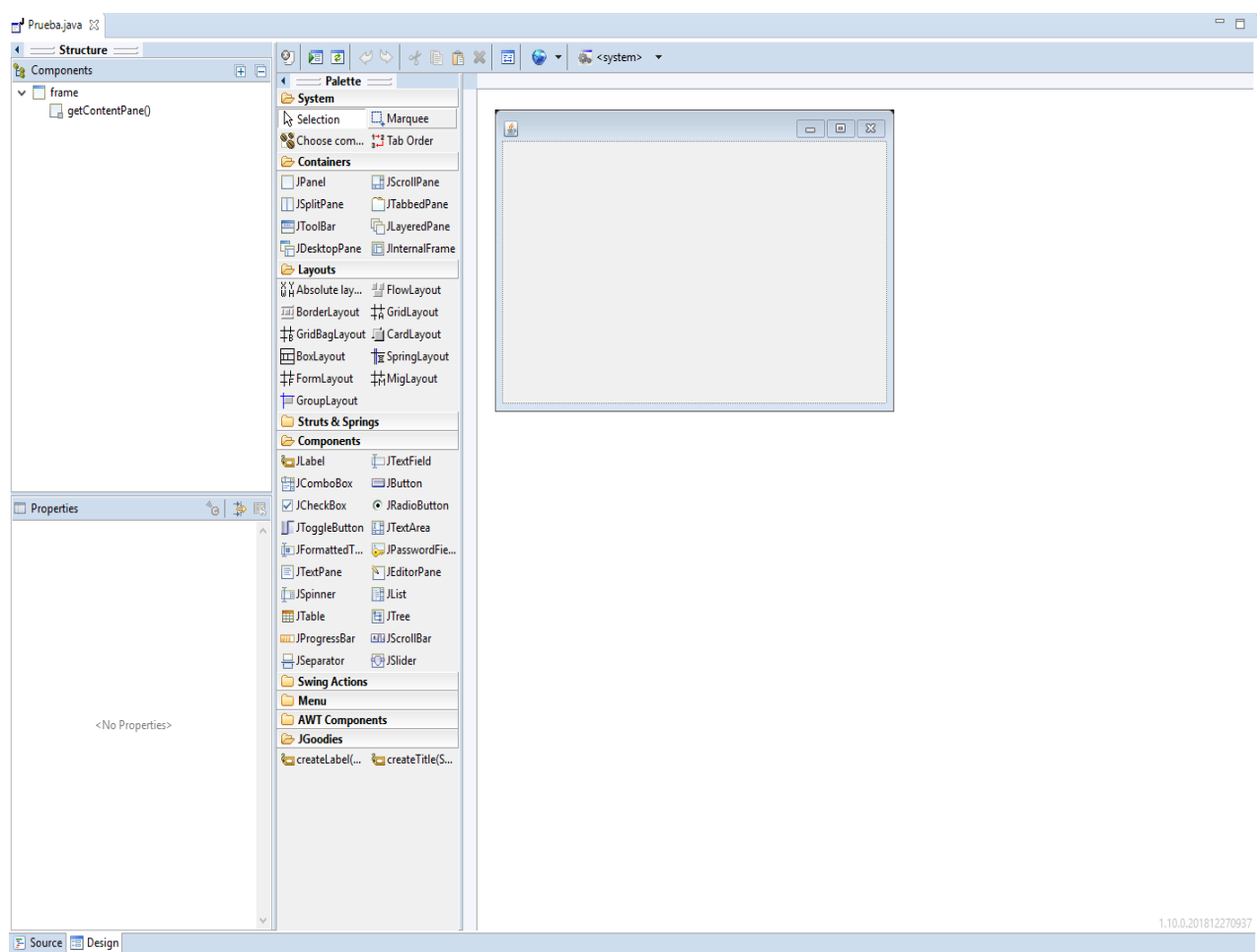
```
1 package pruebaWindowBuilder;
2
3 import java.awt.EventQueue;
4
5
6
7 public class Prueba {
8
9     private JFrame frame;
10
11     /**
12      * Launch the application.
13      */
14     public static void main(String[] args) {
15         EventQueue.invokeLater(new Runnable() {
16             public void run() {
17                 try {
18                     Prueba window = new Prueba();
19                     window.frame.setVisible(true);
20                 } catch (Exception e) {
21                     e.printStackTrace();
22                 }
23             }
24         });
25     }
26
27     /**
28      * Create the application.
29      */
30     public Prueba() {
31         initialize();
32     }
33
34     /**
35      * Initialize the contents of the frame.
36      */
37     private void initialize() {
38         frame = new JFrame();
39         frame.setBounds(100, 100, 450, 300);
40         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
41     }
42
43 }
44
```

Source Design

Esperamos a que aparezca en pantalla.



Nos aparecerá algo como esto en pantalla



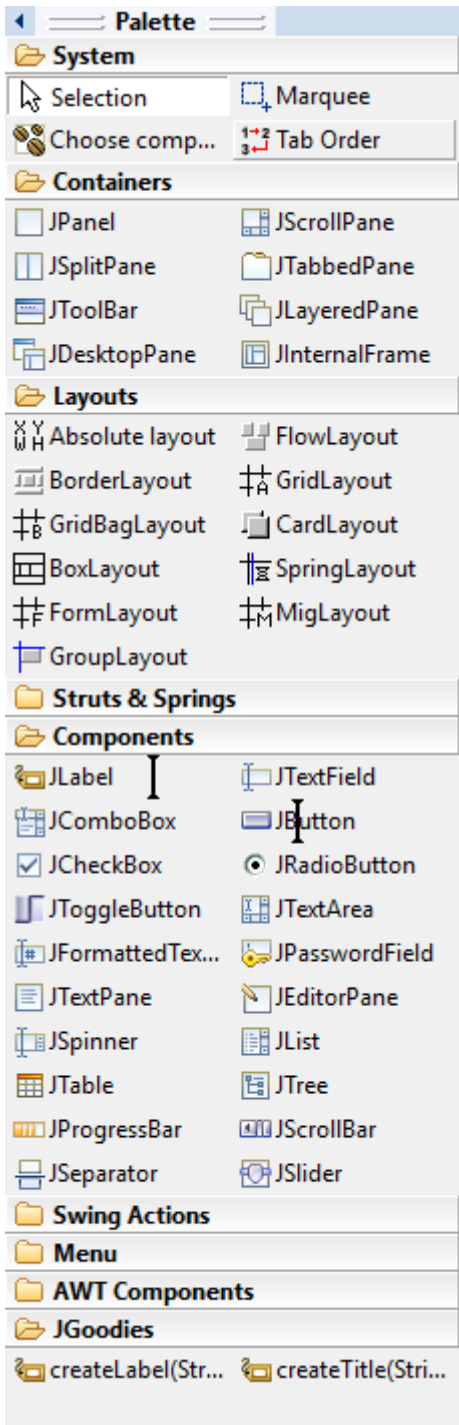
Esta es la ventana principal de WindowBuilder. Como veis, nos permite añadir elementos gráficos como imágenes, botones o listas a nuestro programa, a parte de otros muchos.

Ejemplo práctico:

Vamos a realizar un ejemplo práctico de una mini calculadora con cinco funcionalidades: cuatro operaciones básicas (suma, resta, producto y división) y un botón de reseteo.

Creamos un nuevo proyecto como he explicado anteriormente. En mi caso se lo voy a llamar pruebaCalculadora, y a continuación creo un windowAplication que se llama calculadora.

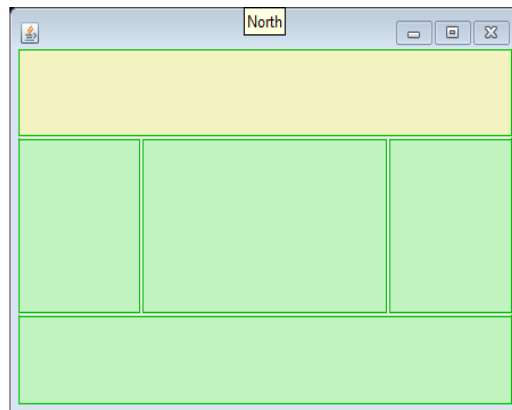
Al volver a observar la interfaz gráfica observamos la paleta que hay a la izquierda de nuestra ventana.



En este ejemplo solo vamos a usar la sección de "System", "Layouts" y "Components".

1º. En la sección de "system" tenemos las elecciones básicas para seleccionar todos los distintos elementos que queremos poner en nuestra ventana. Los más básicos son el "selection" (haciendo clic o arrastrando cada componente hacia la ventana) y "marquee", el cuál sirve para seleccionar más de un elemento a la vez.

2º. En la sección de "layouts" encontraremos diversos modos para poder colocar los distintos elementos que contendrá nuestra ventana. Por defecto, se distribuirá en norte, sur, este, oeste y en el centro.

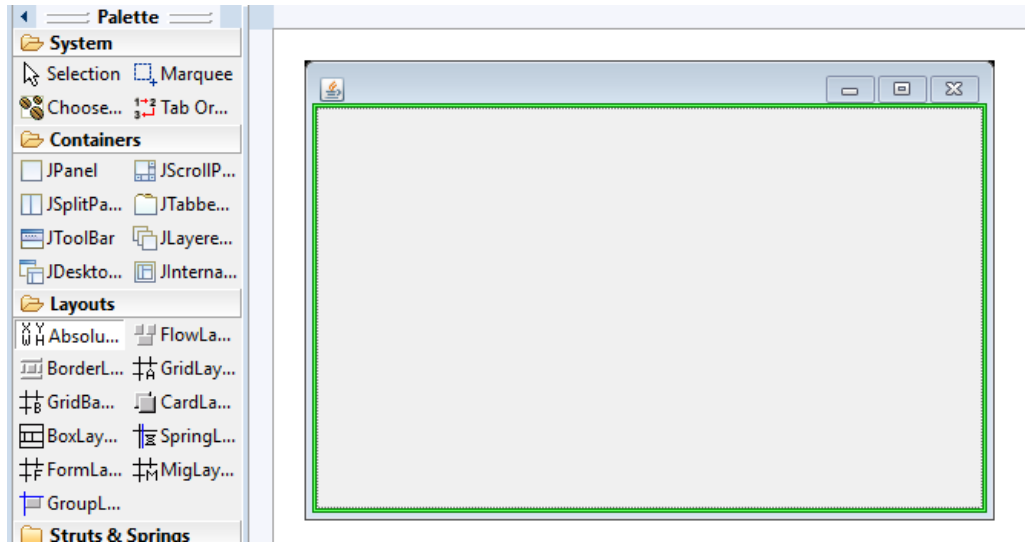


En este ejemplo vamos a usar la opción "Absolute layout". La cuál nos permite colocar los diversos componentes como queramos.

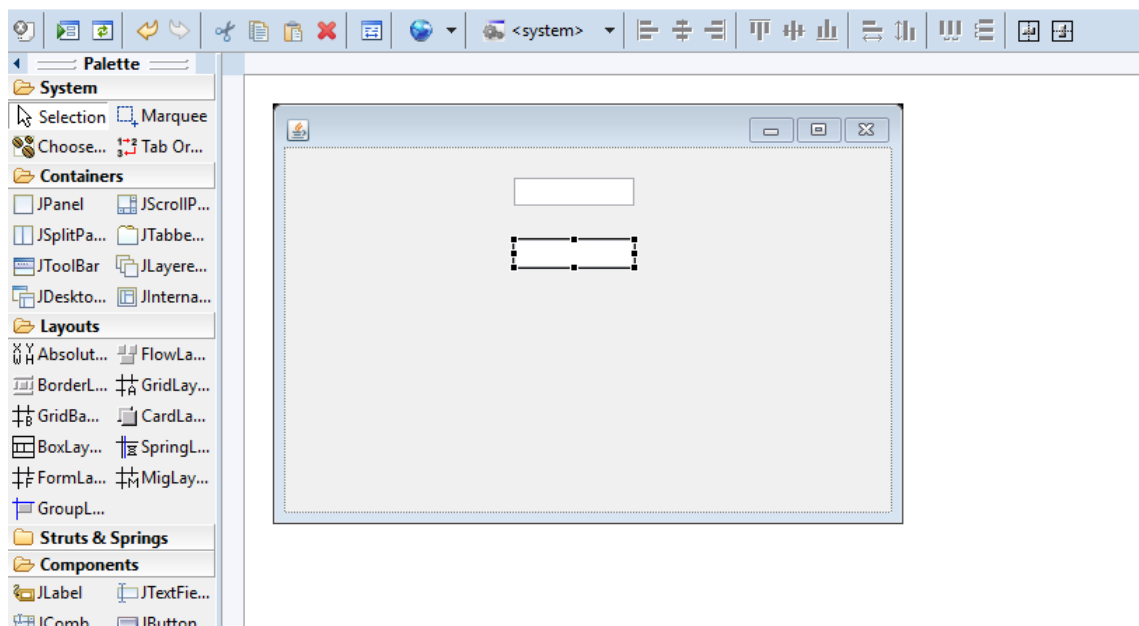
3º. Por último en "components" tenemos todos los distintos elementos que podemos introducir en nuestra ventana. En este ejercicio práctico vamos a usar los "JLabel" (etiquetas), "JTextField" (campos de texto) y "JButton" (botones).

Una vez conocido cómo vamos a trabajar nuestra ventana empezamos a realizar nuestra calculadora.

1º. Elegimos de qué forma vamos a colocar los componentes. En este caso como ya he explicado anteriormente elegimos "Absolute layout".



2º. Añadimos dos "JTextField" (campos de texto) que van a ser los campos donde vamos a introducir los números que el usuario escriba por teclado. Además a la izquierda de estos colocamos dos etiquetas que describan el campo en sí. Con el la opción de selección hacemos clic en el elemento y volvemos a pulsar en la ventana o arrastramos el elemento hacia esta.



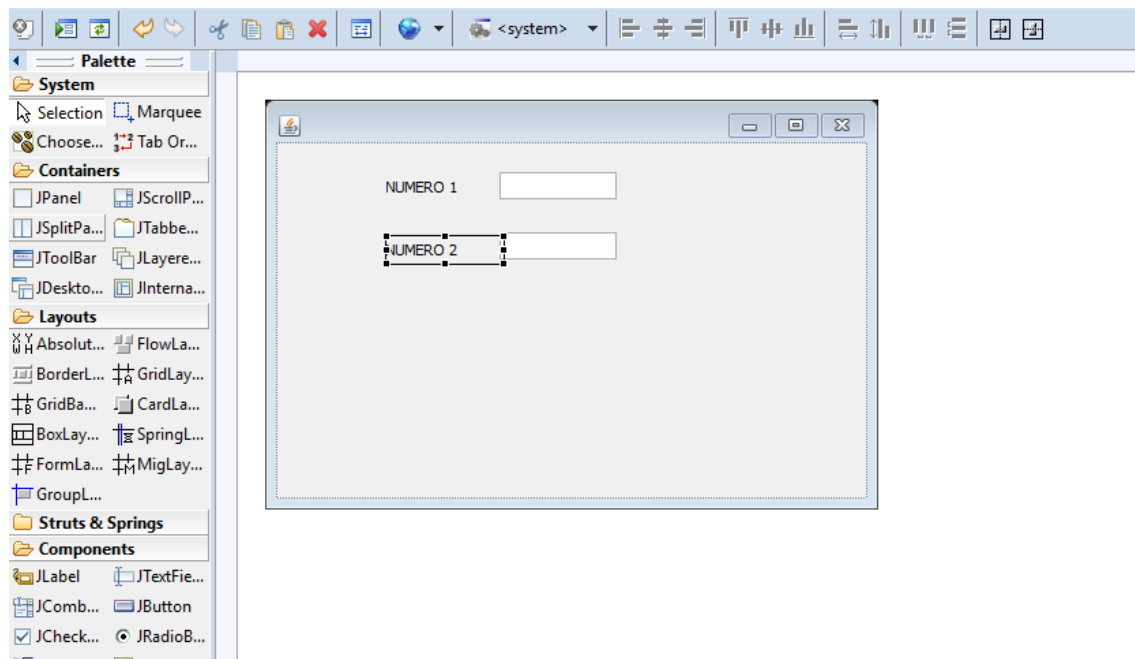
A nuestro se le añadirá lo siguiente:

```
private void initialize() {
    frame = new JFrame();
    frame.setBounds(100, 100, 450, 300);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.getContentPane().setLayout(null);

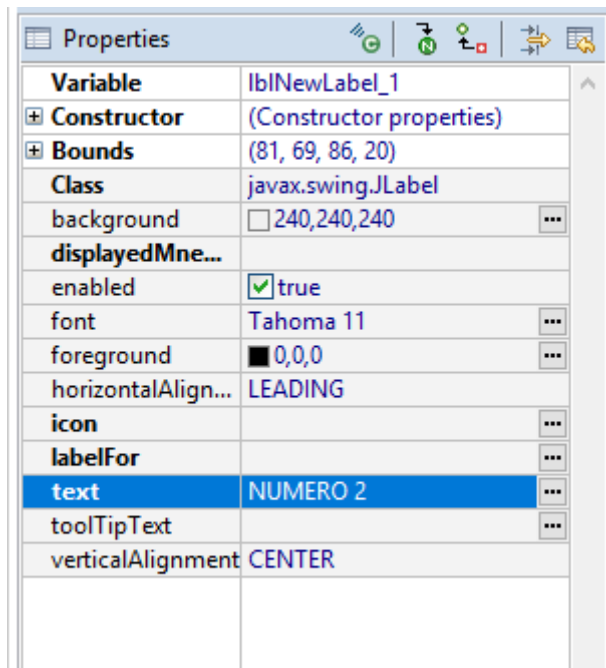
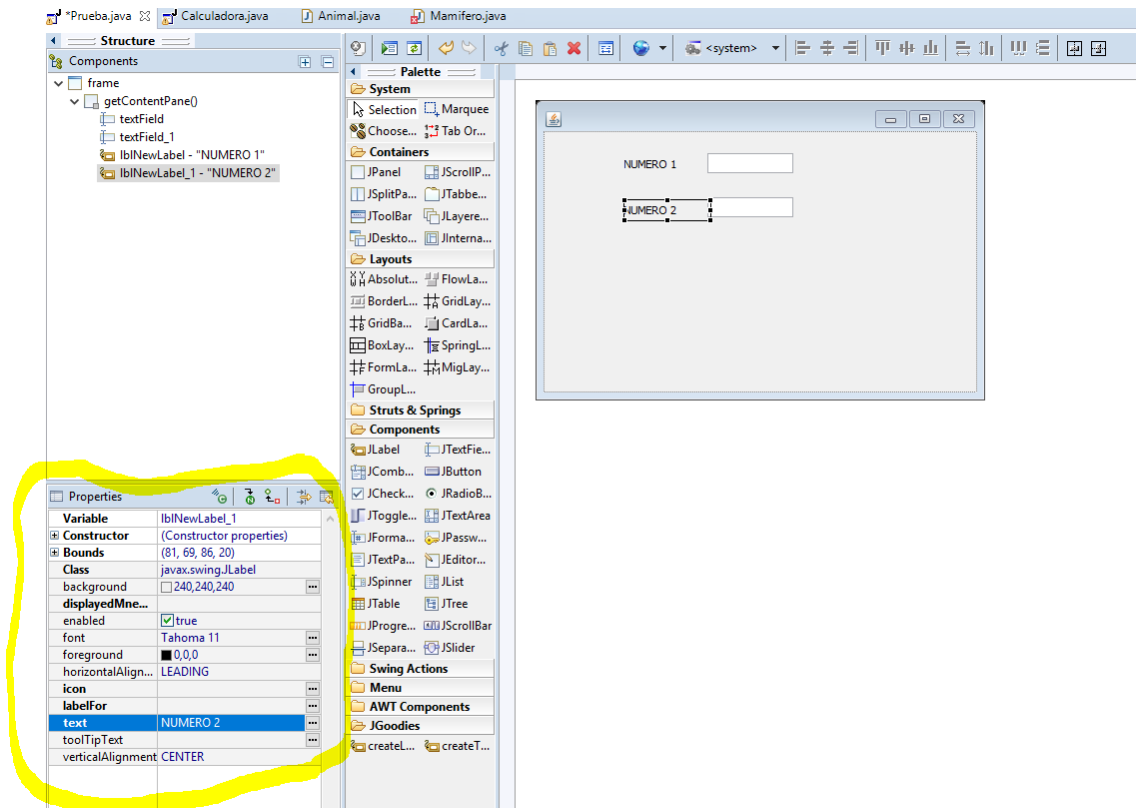
    textField = new JTextField();
    textField.setBounds(164, 22, 86, 20);
    frame.getContentPane().add(textField);
    textField.setColumns(10);

    textField_1 = new JTextField();
    textField_1.setBounds(164, 66, 86, 20);
    frame.getContentPane().add(textField_1);
    textField_1.setColumns(10);
}
```

3º. Ahora vamos a añadir a nuestros cuadros de texto dos etiquetas que nos indican que es cada uno de ellos. Esto es con el elemento "JLabel".



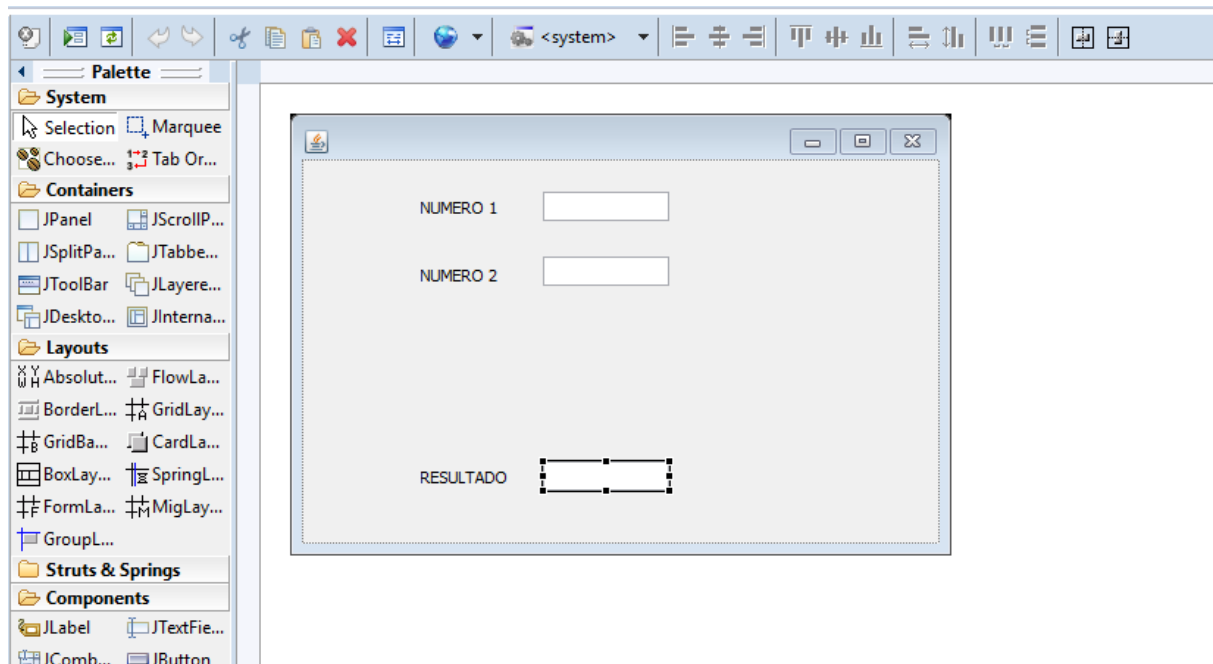
Para colocarlas correctamente recordar que podemos variar su tamaño fácilmente con el cursor moviendo por las esquinas. Para cambiar el nombre de la etiqueta nos vamos a la esquina inferior izquierda donde tenemos la tabla con las propiedades.



En "text" cambiamos el nombre del elemento.

Como podemos ver tenemos distintos apartados de nuestro elemento en cuestión. En este ejemplo solo vamos a modificar "text" y "variable" (lo editaremos para aclarar más el código).

Muy bien, ahora vamos a introducir de igual manera que hemos hecho hasta ahora el cuadro de texto y la etiqueta con el resultado.



Bien, ahora para aclarar más el código vamos a cambiar el nombre de las variables a los siguientes : numero1, numero2, resultado, cuadroNum1, cuadroNum2 y cuadroResultado. El código nos quedaría así:

```
package pruebaWindowBuilder;

import java.awt.EventQueue;

import javax.swing.JFrame;
import javax.swing.JTextField;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JButton;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
import java.awt.BorderLayout;

public class Prueba {

    private JFrame frame;
    private JTextField cuadroNum1;
    private JTextField cuadroNum2;
    private JTextField cuadroResultado;
    /**
     * Lanza la aplicación.
     */
    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
```

```
        try {
            Prueba window = new Prueba();
            window.frame.setVisible(true);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
});
}
/**
 * Creación de la aplicación.
 */
public Prueba() {
    initialize();
}
/**
 * Inicializar contenidos de la ventana.
 */
private void initialize() {
    frame = new JFrame();
    frame.setBounds(100, 100, 450, 300);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.getContentPane().setLayout(null);

    cuadroNum1 = new JTextField();
    cuadroNum1.setBounds(164, 22, 86, 20);
    frame.getContentPane().add(cuadroNum1);
    cuadroNum1.setColumns(10);

    cuadroNum2 = new JTextField();
    cuadroNum2.setBounds(164, 66, 86, 20);
    frame.getContentPane().add(cuadroNum2);
    cuadroNum2.setColumns(10);

    JLabel numero1 = new JLabel("NUMERO 1");
    numero1.setBounds(81, 25, 86, 17);
    frame.getContentPane().add(numero1);

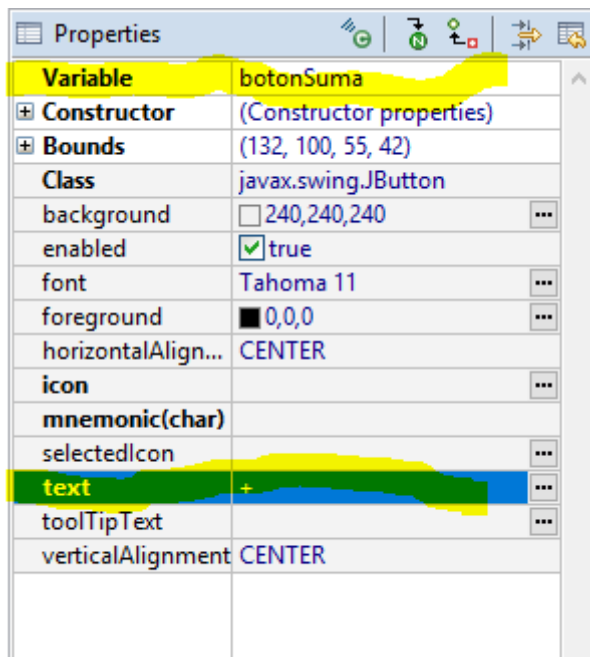
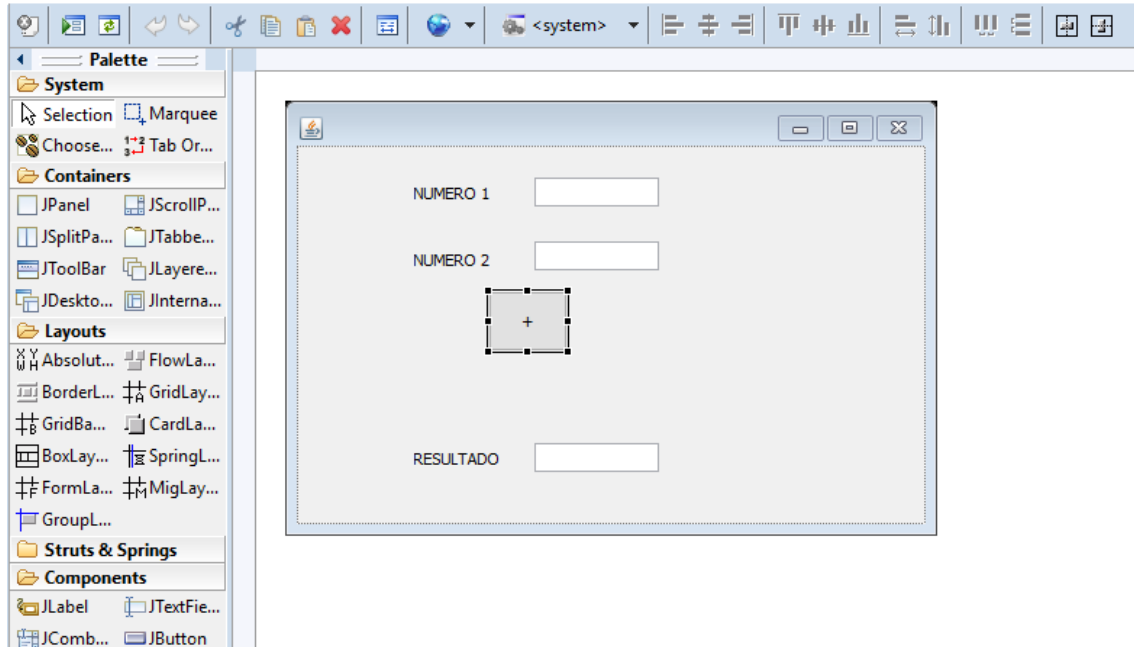
    JLabel numero2 = new JLabel("NUMERO 2");
    numero2.setBounds(81, 69, 86, 20);
    frame.getContentPane().add(numero2);

    cuadroResultado = new JTextField();
    cuadroResultado.setBounds(164, 205, 86, 20);
    frame.getContentPane().add(cuadroResultado);
    cuadroResultado.setColumns(10);

    JLabel resultado = new JLabel("RESULTADO");
    resultado.setBounds(81, 208, 80, 17);
    frame.getContentPane().add(resultado);
}
}
```

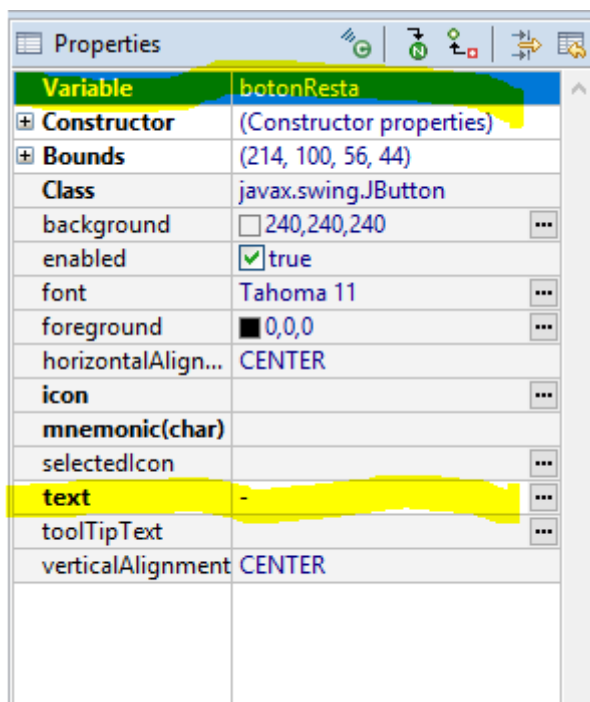
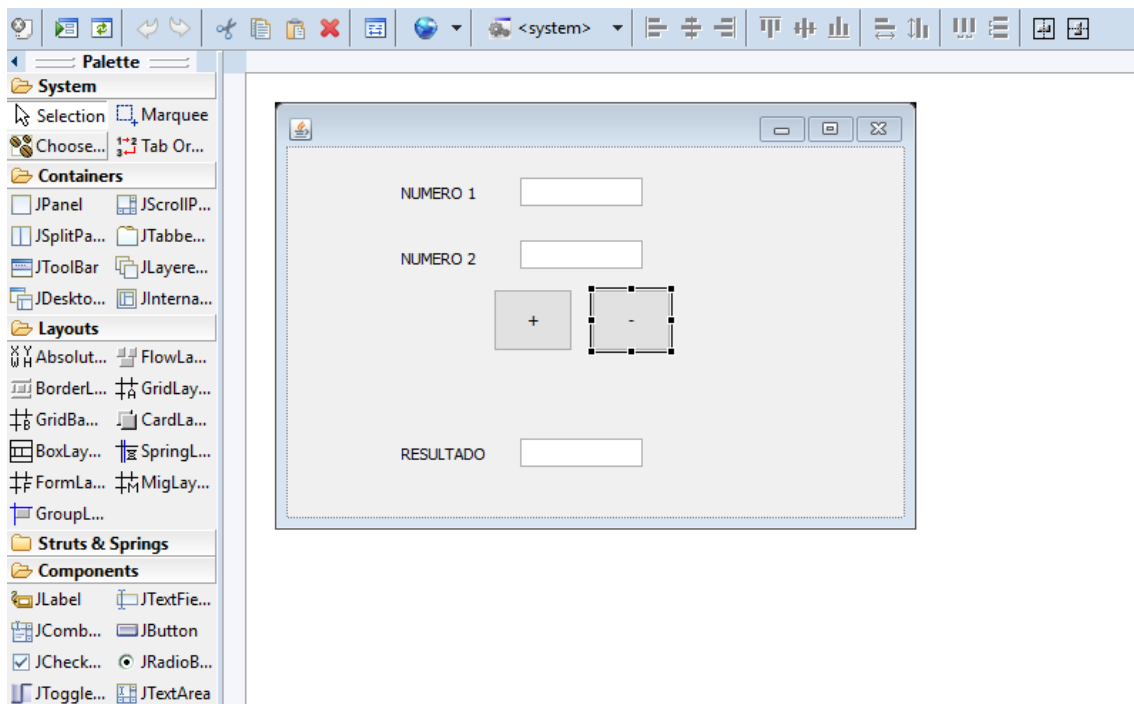
4º. Ahora nos queda añadir los botones de nuestra calculadora. Para ello añadiremos cuatro "JButton" para las cuatro operaciones y uno más para resetear los cuadros de texto. Luego les daremos la funcionalidad escribiendo el código a cada uno de nuestros botones.

BOTÓN SUMA



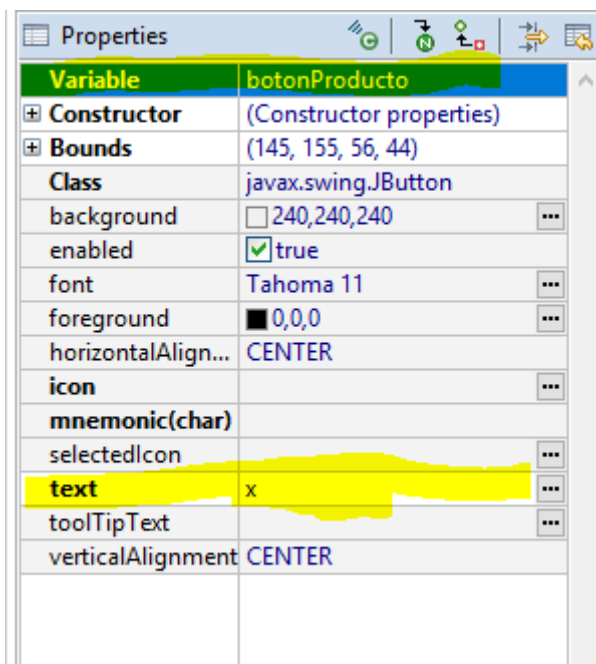
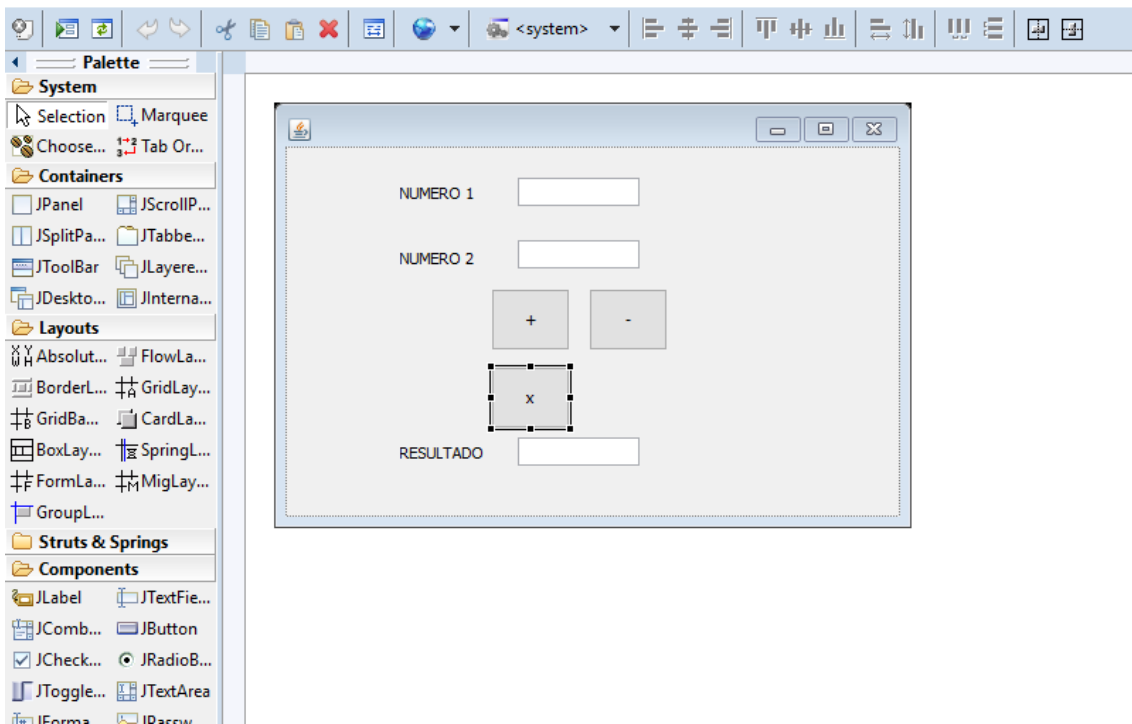
Teniendo en cuenta la imagen anterior, aquí cambiaremos el nombre de la variable y el nombre que aparecerá dentro del botón que hemos creado.

BOTÓN RESTA



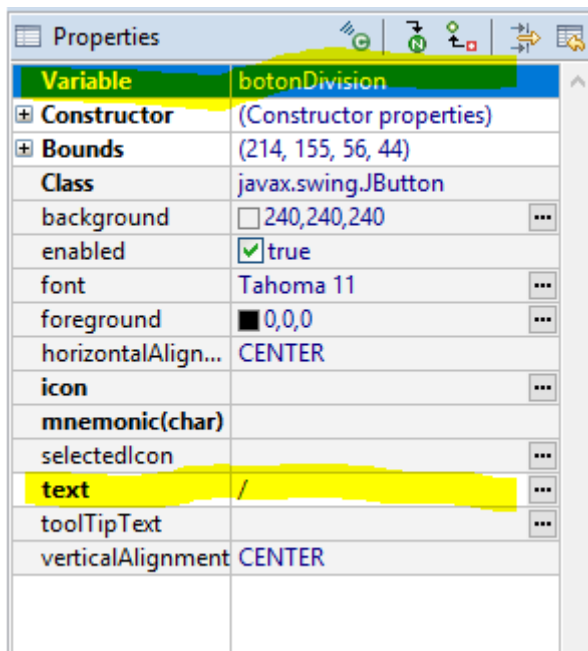
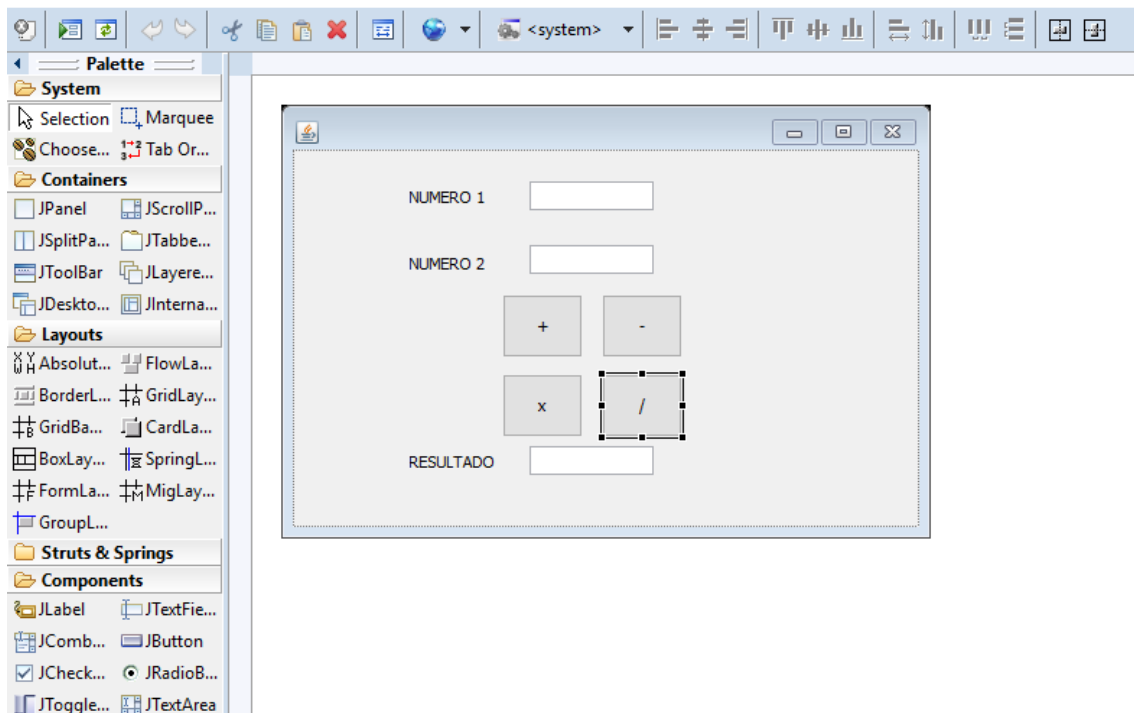
Teniendo en cuenta la imagen anterior, aquí cambiaremos el nombre de la variable y el nombre que aparecerá dentro del botón que hemos creado.

BOTÓN MULTIPLICACION



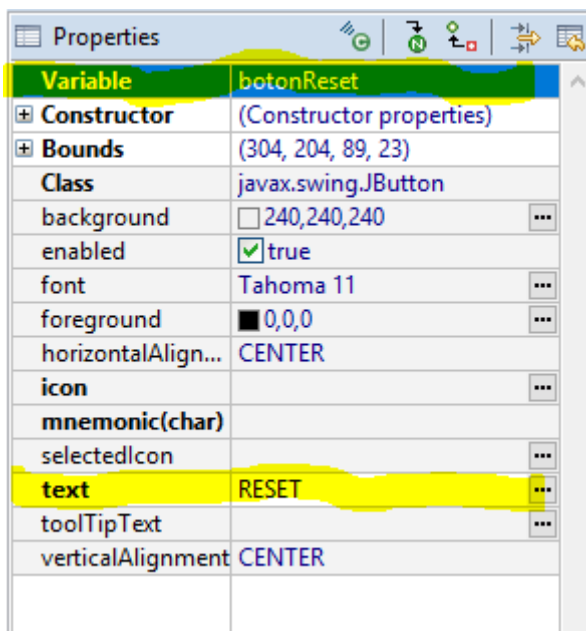
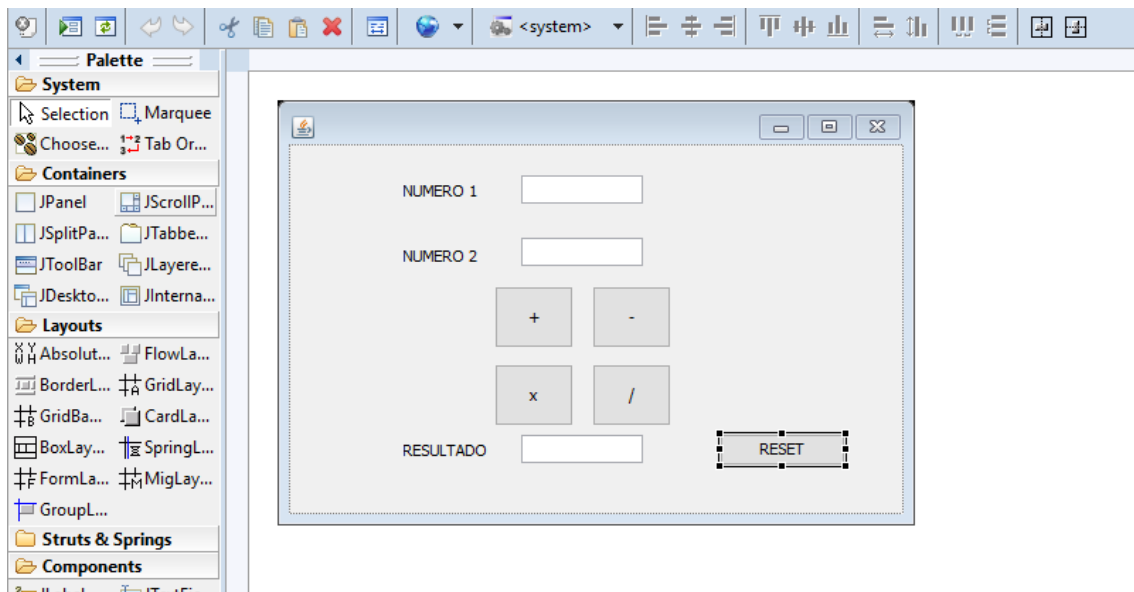
Teniendo en cuenta la imagen anterior, aquí cambiaremos el nombre de la variable y el nombre que aparecerá dentro del botón que hemos creado.

BOTÓN DIVISION



Teniendo en cuenta la imagen anterior, aquí cambiaremos el nombre de la variable y el nombre que aparecerá dentro del botón que hemos creado.

BOTÓN RESET



Teniendo en cuenta la imagen anterior, aquí cambiaremos el nombre de la variable y el nombre que aparecerá dentro del botón que hemos creado.

5º. Una vez creados los botones nos queda añadirles la lógica en el código, para ello hacemos doble clic al botón que queramos y nos aparecerá lo siguiente:

```

JButton botonSuma = new JButton("+");
    botonSuma.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent arg0) {
            //Aquí escribimos el código
        }
    });

```

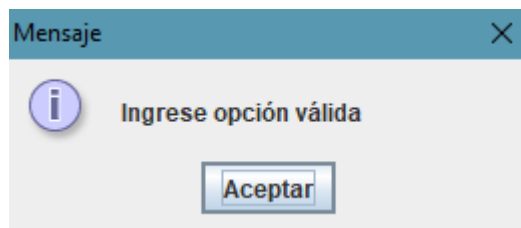
Por ejemplo en el botón suma escribiremos lo siguiente:

```

JButton botonSuma = new JButton("+");
    botonSuma.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent arg0) {
            Integer num1, num2, suma;
            try {
                num1 = Integer.parseInt(numero1.getText());
                num2 = Integer.parseInt(numero2.getText());
                suma = num1 + num2;
                resultado.setText(Integer.toString(suma));
            } catch (Exception e1) {
                JOptionPane.showMessageDialog(null, "Ingrese
                opción válida");
            }
        }
    });

```

Cuando el usuario no introduzca un numero entero saltara una excepción. Saldrá la siguiente ventana de alerta.:



La lógica con el resto de botones es similar a la de la suma:

```

JButton botonResta = new JButton("-");
    botonResta.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            Integer num1, num2, resta;
            try {
                num1 = Integer.parseInt(numero1.getText());
                num2 = Integer.parseInt(numero2.getText());
                resta = num1 - num2;
                resultado.setText(Integer.toString(resta));
            } catch (Exception e1) {
                JOptionPane.showMessageDialog(null, "Ingrese
                opción válida");
            }
        }
    });
    botonResta.setBounds(224, 90, 62, 48);
    //Esta parte hace referencia a las dimensiones que le hemos
    //dado al botón que aparece de forma automática
    frame.getContentPane().add(botonResta);
    //Esta parte es la visualización del botón en la ventana

```

```
JButton botonMultiplicacion = new JButton("x");
    botonMultiplicacion.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            Integer num1, num2, producto;
            try {
                num1 = Integer.parseInt(numero1.getText());
                num2 = Integer.parseInt(numero2.getText());
                producto = num1 * num2;
                resultado.setText(Integer.toString(producto));
            } catch (Exception e1) {
                JOptionPane.showMessageDialog(null, "Ingrese
                opción válida");
            }
        }
    });
    botonMultiplicacion.setBounds(161, 139, 62, 48);
    frame.getContentPane().add(botonMultiplicacion);

JButton botonDivision = new JButton("/");
    botonDivision.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            Integer num1, num2, division;
            try {
                num1 = Integer.parseInt(numero1.getText());
                num2 = Integer.parseInt(numero2.getText());
                division = num1 / num2;
                resultado.setText(Integer.toString(division));
            } catch (Exception e1) {
                JOptionPane.showMessageDialog(null, "Ingrese
                opción válida");
            }
        }
    });
    botonDivision.setBounds(224, 139, 62, 48);
    frame.getContentPane().add(botonDivision);
```

Por último el código del botón reset quedaría así:

```
JButton botonReset = new JButton("RESET");
    botonReset.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            numero1.setText(null);
            numero2.setText(null);
            resultado.setText(null);
        }
    });
    botonReset.setBounds(302, 197, 89, 23);
    frame.getContentPane().add(botonReset);
```

Su funcionamiento se basa en cambiar el valor a nulo en los tres cuadros de texto (resetear).

6°. Y ya tendríamos acabada nuestra calculadora. ¡Ahora solo queda ejecutarlo y probarla!
El código definitivo quedaría algo así:

```
package calculadora;

import java.awt.EventQueue;

import javax.swing.JFrame;
import java.awt.BorderLayout;
import javax.swing.JTextField;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import java.awt.GridBagLayout;
import java.awt.GridBagConstraints;
import java.awt.FlowLayout;
import javax.swing.JLabel;
import javax.swing.JOptionPane;

import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

public class Calculadora {

    private JFrame frame;
    private JTextField numero1;
    private JTextField numero2;
    private JTextField resultado;

    /**
     * Launch the application.
     */
    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    Calculadora window = new Calculadora();
                    window.frame.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }

    /**
     * Creación de la aplicación.
     */
    public Calculadora() {
        initialize();
    }
}
```

```
/**
 * Inicialización y creación de los botones.
 */

private void initialize() {

    frame = new JFrame();
    frame.setBounds(100, 100, 450, 300);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.getContentPane().setLayout(null);

    numero1 = new JTextField();
    numero1.setBounds(172, 28, 86, 20);
    frame.getContentPane().add(numero1);
    numero1.setColumns(10);

    numero2 = new JTextField();
    numero2.setBounds(172, 59, 86, 20);
    frame.getContentPane().add(numero2);
    numero2.setColumns(10);

    JLabel lblNewLabel = new JLabel("NUMERO 1");
    lblNewLabel.setBounds(89, 31, 86, 17);
    frame.getContentPane().add(lblNewLabel);

    JLabel lblNewLabel_1 = new JLabel("NUMERO 2");
    lblNewLabel_1.setBounds(89, 62, 86, 17);
    frame.getContentPane().add(lblNewLabel_1);

    JButton botonSuma = new JButton("+");
    botonSuma.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent arg0) {
            Integer num1, num2, suma;
            try {
                num1 = Integer.parseInt(numero1.getText());
                num2 = Integer.parseInt(numero2.getText());
                suma = num1 + num2;
                resultado.setText(Integer.toString(suma));
            } catch (Exception e1) {
                JOptionPane.showMessageDialog(null,
                    "Ingrese opción válida");
            }
        }
    });

    botonSuma.setBounds(161, 90, 62, 48);
    frame.getContentPane().add(botonSuma);

    JButton botonResta = new JButton("-");
```

```
        botonResta.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                Integer num1, num2, resta;
                try {
                    num1 = Integer.parseInt(numero1.getText());
                    num2 = Integer.parseInt(numero2.getText());
                    resta = num1 - num2;
                    resultado.setText(Integer.toString(resta));
                } catch (Exception e1) {
                    JOptionPane.showMessageDialog(null,
                        "Ingrese opción válida");
                }
            }
        });
        botonResta.setBounds(224, 90, 62, 48);
        frame.getContentPane().add(botonResta);

        JButton botonMultiplicacion = new JButton("x");
        botonMultiplicacion.addActionListener(new
        ActionListener() {
            public void actionPerformed(ActionEvent e) {
                Integer num1, num2, producto;
                try {
                    num1 = Integer.parseInt(numero1.getText());
                    num2 = Integer.parseInt(numero2.getText());
                    producto = num1 * num2;
                    resultado.setText(Integer.toString(producto));
                } catch (Exception e1) {
                    JOptionPane.showMessageDialog(null,
                        "Ingrese opción válida");
                }
            }
        });

        botonMultiplicacion.setBounds(161, 139, 62, 48);
        frame.getContentPane().add(botonMultiplicacion);

        JButton botonDivision = new JButton("/");
        botonDivision.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                Integer num1, num2, division;

                try {
                    num1 = Integer.parseInt(numero1.getText());
                    num2 = Integer.parseInt(numero2.getText());
                    division = num1 / num2;
                    resultado.setText(Integer.toString(division));
                } catch (Exception e1) {
                    JOptionPane.showMessageDialog(null,
                        "Ingrese opción válida");
                }
            }
        });

        botonDivision.setBounds(224, 139, 62, 48);
        frame.getContentPane().add(botonDivision);
```

```
        JButton botonReset = new JButton("RESET");
        botonReset.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                numero1.setText(null);
                numero2.setText(null);
                resultado.setText(null);
            }
        });
        botonReset.setBounds(302, 197, 89, 23);
        frame.getContentPane().add(botonReset);

        resultado = new JTextField();
        resultado.setBounds(172, 198, 86, 20);
        frame.getContentPane().add(resultado);
        resultado.setColumns(10);

        JLabel lblNewLabel_2 = new JLabel("RESULTADO");
        lblNewLabel_2.setBounds(89, 201, 86, 17);
        frame.getContentPane().add(lblNewLabel_2);
    }
}
```


Solución de problemas y preguntas frecuentes

El problema principal y más grave que podemos encontrarnos referente al plugin es que después de instalar windowbuilder, tengamos dificultades a la hora de crear un proyecto o no nos aparezca que se ha instalado por ninguna parte. Esto es debido a problemas de compatibilidad entre el plugin y eclipse. Por lo que para evitarnos este tipo de errores tendremos que confirmar que las versiones de eclipse y java son compatibles con la versión actual de windowbuilder (véase *Requisitos*).

P. ¿Cómo puedo empezar a crear aplicaciones de ventana?

R. Véase *Manejo de funciones básicas*.

P. ¿Cómo accedo a la interfaz gráfica de mi aplicación de ventana?

R. En la pestaña "Design" dentro de la ventana.

P. ¿Por qué es tan limitada la colocación de los elementos a mi antojo?

R. Porque no estará usando el layout adecuado.

P. ¿Cómo puedo dar funcionalidades a los botones que he colocado?

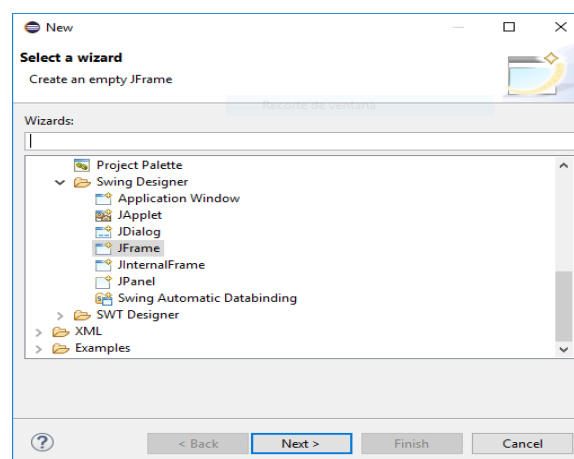
R. Haciendo doble click sobre el botón y añadir el código más tarde.

P. ¿Qué es el código "basura"?

R. Básicamente es código que se genera automáticamente al colocar un componente en nuestra ventana el cuál muchas veces carece de utilidad ya que podemos prescindir de él.

P. ¿Es lo mismo un JFrame, que un JDialog, que un JPanel, que una WindowApplication?

R. A la hora de crear una aplicación vimos que en la carpeta de "swing designer" tenemos multitud de variaciones por ejemplo: un JFrame es una ventana, un JPanel es un contenedor de componentes de interfaz grafica, un JFrame puede contener muchos JPanel y un JPanel puede contener muchos JTextField, JButton, JLabel, JComboBox, etc., Entonces usas un JFrame para definir una ventana y usas JPanel para añadir controles GUI al JFrame.



Glosario

Absolute layout: Distribución que podemos dar a nuestra ventana para que los componentes que coloquemos en ella los podamos distribuir libremente.

EPL (pag 7 *Requisitos mínimos del sistema*): Significa Licencia Pública Eclipse. Es una licencia de software de código abierto utilizada por la Fundación Eclipse para su software. Sustituye a la Licencia Pública Común y elimina ciertas condiciones relativas a los litigios sobre patentes.

Holger Voormann (pag 7 *Requisitos mínimos del sistema*): Holger Voormann es un desarrollador de software, consultor, ávido usuario de Eclipse y colaborador de Eclipse. Su interés está en el diseño de interfaces de usuario, control de calidad y procesos ágiles.

IDE: Entorno de desarrollo.

Interfaz gráfica: La interfaz de usuario es la parte del programa que permite al usuario interactuar con él.

JButton: botón al que le daremos alguna funcionalidad

JLabel: etiqueta

TextField: campo de texto

Layout: Diseño, en este caso aplicado al propio JFrame.

Marketplace: Tienda de plugins del IDE de Eclipse.

Photon, Oxygen, Neon: Son distintas distribuciones de Eclipse.

Plugin: Extensión de un IDE que incorpora o mejora nuevas funcionalidades.

Programación orientada a Eventos: Paradigma de programación determinada por los sucesos ocurridos en el sistema.

Wizard: Cualquier programa intuitivo que guía paso a paso al usuario para realizar una tarea. El ayudante de Office es un ejemplo.

Webgrafía

Información sobre WindowBuilder:

<https://www.espai.es/blog/2014/05/disenio-de-interfaces-en-java-con-windowbuilder/>

<http://www.tutorialesprogramacionya.com/javaya/detalleconcepto.php?codigo=128&punto=&inicio=>

Descarga e instalación del IDE Eclipse Oxygen:

<https://jarroba.com/instalar-bien-eclipse-un-ide-de-muchos/>

Guia oficial de Window Builder

<https://help.eclipse.org/mars/index.jsp?topic=%2Forg.eclipse.wb.doc.user%2Fhtml%2Findex.html&cp=79>

Video tutorial de como hacer una calculadora con WindowBuilder

<https://www.youtube.com/watch?v=dGw9BfCh5SU>